

Project 1 of Sistemi e architetture per Big Data

Comparetto Matthieu
Erasmus student
Tor Vergata Engineering Department
Toulouse, France
commatthieu@gmail.com

Abstract—This paper presents the design and implementation of a distributed Big Data architecture aimed at analyzing large-scale flight data across multiple airlines. To simulate a real-world distributed environment on a single machine, we utilized Docker to deploy a Hadoop Distributed File System (HDFS) comprising a NameNode and a DataNode, alongside an Apache Spark cluster consisting of one Master and two Workers. The analytical queries were developed in Java leveraging the Spark Dataset API for optimized distributed processing. Finally, a performance evaluation was conducted on the local host, with execution times and system metrics processed and visualized using Python (Pandas and Matplotlib).

I. INTRODUCTION

The airline industry generates huge amounts of data every day. Organizations like the US Department of Transportation share large datasets containing details about flight delays and cancellations. Analyzing this much information—specifically 2.2 million flight records from January to April 2025—is simply too heavy for traditional databases. To process this data efficiently, we need to use distributed computing tools. The main challenge is extracting useful patterns from this large-scale dataset. We need to calculate complex statistics, such as average delays, the impact of weather or security, delay percentiles across millions of rows, and even perform a K-means clustering algorithm on normalized data. Moreover, setting up a reliable distributed environment to test these operations on a single computer is a technical challenge on its own. To solve these problems for the Sistemi e Architetture per Big Data course, this project presents a fully containerized architecture. We used Docker Compose to build a simulated cluster on a single machine. First, we loaded the data into a Hadoop Distributed File System (HDFS). Then, we processed it using an Apache Spark cluster. We wrote several queries using Spark DataFrames, strictly avoiding Spark SQL. These queries analyzed monthly cancellation rates, computed hourly delay percentiles, and performed a k-means clustering analysis on different airlines. Finally, we measured the execution times to evaluate the platform's performance. The rest of this paper is organized as follows: Section II explains the system architecture and data ingestion. Section III describes how we implemented the Spark queries. Section IV shows the graphs and discusses the performance results. Finally, Section V concludes the project.

II. MAIN ARCHITECTURE

To simulate a distributed Big Data environment on a single machine, we orchestrated our infrastructure using Docker Compose. Our docker-compose.yml file is divided into two primary layers: storage and processing. For the storage layer, we deployed a Hadoop Distributed File System (HDFS) consisting of one NameNode and one dependent DataNode. For the computation layer, we built an Apache Spark 3.3.0 cluster featuring one Master node and two Worker nodes. To ensure efficient parallel processing, each Spark worker is explicitly configured with 4 CPU cores and 3 GB of memory (based on my own personal computer for performance). Furthermore, we used Docker volumes to map local directories (./data, ./results, and ./target) directly into the containers, allowing seamless access to datasets, compiled applications, and output results without manual copying. The project's build process and libraries are managed with Maven through the pom.xml file. To match our cluster environment, we imported the spark sql 2.12 library (version 3.3.0) for Dataset API manipulations, as well as the sparkmllib 2.12 package to implement the k-means clustering algorithm required for our analysis. We configured the maven-assembly-plugin to compile our application into a "fat JAR". This approach packages our compiled code alongside all necessary dependencies into a single executable file, greatly simplifying the deployment to the Spark Master node. At the software level, our Java application is structured to strictly separate data preparation from analytical logic. The Main class acts as the central orchestrator: it initializes the SparkSession, parses command-line arguments to determine which query the user wants to execute, and routes the execution flow accordingly. Before any query is executed, the Main class delegates the ingestion phase to a dedicated DataLoader class. The DataLoader is responsible for connecting to HDFS, reading the raw CSV files, handling missing or corrupted values, and mapping the raw strings into a typed Spark Dataset. By isolating the ingestion logic within the DataLoader, our specific query classes receive clean, structured, and ready-to-process data, making the overall codebase much more modular and easier to maintain. Also, the Main class is designed to execute the selected queries over multiple iterations (three by default). It records the execution time for each loop and computes the overall average. The results of each query are exported directly in HDFS using the SaveMode.Overwrite

III. IMPLEMENTATION OF THE SPARK QUERIES

To process the dataset efficiently, we developed four analytical queries using the typed Spark Dataset API. Following the project guidelines, we exclusively used DataFrames transformations, completely avoiding Spark SQL.

A. Query 1

The first query evaluates the performance of American Airlines (AA) and Delta (DL). Using the `filter()` function coupled with `col().equalTo()`, we isolate the relevant carriers and immediately persist the subset using `.cache()`. This caching is crucial because the DataFrame is split into two logical branches. By spitting the following Dataset into two branches, we don't need to fill the missing value for the delay by 0. In the first branch, we use `filter()` with `isNotNull()` and `.equalTo(0.0)` on the "CANCELLED" column to isolate actual flights. We then use `groupBy("MONTH", "OP_UNIQUE_CARRIER")` and chain the `agg()` function with `avg()`, `min()`, and `max()` to compute delay statistics. In the second branch, working on the cached dataset that includes cancelled flights, we compute the cancellation rate using `sum()`, `.divide()`, and `.multiply(100)` within the `agg()` function. Finally, we merge the branches using an inner `join()` on the month and carrier columns, structure the final dataset with `select()` (targeting metrics like `dep_delay_mean` and `cancellation_rate`), release the memory using `.unpersist()`, and sort the output with `orderBy()`.

B. Query 2

This query identifies the airlines with the highest average arrival delays and breaks down the delay components. We start by cleaning the data with a strict `filter()` ensuring both "CANCELLED" and "DIVERTED" columns are equal to 0. The valid flights are then subjected to a `groupBy("OP_UNIQUE_CARRIER")` transformation. Inside the subsequent `agg()` block, we use `count("*")` to compute the total flight volume, alongside multiple `avg()` functions to calculate the mean arrival, departure, carrier, weather, NAS, security, and late aircraft delays (e.g., `avg_arr_delay`, `avg_carrier_delay`). To guarantee the statistical relevance of our findings, we apply a subsequent `filter()` utilizing `.geq(500)` on the computed total flight volume. We finally chain `orderBy()` with `.desc()` on the arrival delay column and restrict the output with `.limit(10)` to extract the top 10 airlines.

C. Query 3

Query 3 analyzes how departure delays fluctuate throughout the day for AA, DL, UA, and WN. After filtering these carriers and applying `.cache()`, we create a new categorical feature. Using `withColumn()`, we divide the "CRS_DEP_TIME" by 100 and chain `.cast(DataTypes.IntegerType)` to truncate the minutes and extract the hour (take the round of the number). Since computing exact percentiles on distributed Big Data is computationally prohibitive, we group the data by carrier and hour, and utilize the `percentile_approx()` (part of the Spark API) function within our aggregations. By passing target percentiles (0.25, 0.50, 0.75, 0.90) and an accuracy parameter

of 1000 via the `lit()` function, Spark efficiently approximates the distribution without triggering memory exceptions. In a separate operation, we group non-cancelled flights by carrier to compute the absolute `min()` and `max()` delays, saving both DataFrames into distinct HDFS directories.

D. Query 4

The final query transitions from standard data aggregation to unsupervised machine learning, requiring the integration of the `org.apache.spark.ml` library. While the standard Spark API is optimized for data manipulation, MLlib is specifically required to apply distributed mathematical models and advanced feature engineering techniques on large-scale data. We begin by cleaning the data using `.na().fill(0.0)` across relevant delay and cancellation columns to prevent algorithmic failures. We then isolate the top 15 airlines by volume and compute their operational metrics. To prepare this data for the algorithm, we use a `VectorAssembler` to concatenate the 8 numerical metrics into a single vector features column. Because these metrics possess vastly different scales (e.g., raw delay minutes versus percentage rates), we apply a `StandardScaler` (configured with `.setWithMean(true)` and `.setWithStd(true)`) to normalize the variables into a `scaled_features` vector. This critical step ensures that no single metric disproportionately influences the distance calculations of the K-Means algorithm. Finally, we instantiate the `KMeans` model. The number of clusters was explicitly set to two (`.setK(2)`). This optimal value was determined through empirical testing: experiments with three clusters ($k = 3$) did not yield significantly more informative or distinct groupings, making two clusters the most interpretable and robust choice for segmenting the airlines' operational behaviors. The model is trained using `.fit()` and applied via `.transform()` to assign a cluster prediction to each airline.

IV. GRAPHS

After executing the distributed computations on our Spark cluster, the aggregated outputs were extracted from local disk (results were also stored in the local disk after the query) in CSV format. To interpret these results, we developed a dedicated data visualization pipeline in Python. This script leverages the `pandas` library for data manipulation and `matplotlib` for generating the visualizations. Furthermore, to effectively visualize the 8-dimensional results of the K-Means clustering algorithm, we utilized the `scikit-learn` library to perform a Principal Component Analysis (PCA), reducing the feature space to two primary dimensions for graphical representation (but more than 91% of the information were saved).

A. Query 1

The first analysis compares the monthly operational performance of American Airlines (AA) and Delta (DL). The results indicate diverging trends in departure delays over the observed four-month period. While DL managed to reduce its average departure delay from approximately 13 minutes in month 1 to under 9 minutes by month 4, AA experienced a

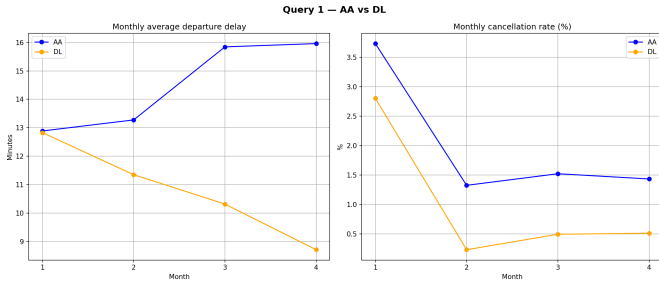


Fig. 1. Comparison of monthly average departure delays and cancellation rates between American Airlines (AA) and Delta (DL).

steady increase, peaking near 16 minutes. Interestingly, both carriers experienced their highest cancellation rates during the first month (3.7% for AA and 2.8% for DL).

B. Query 2

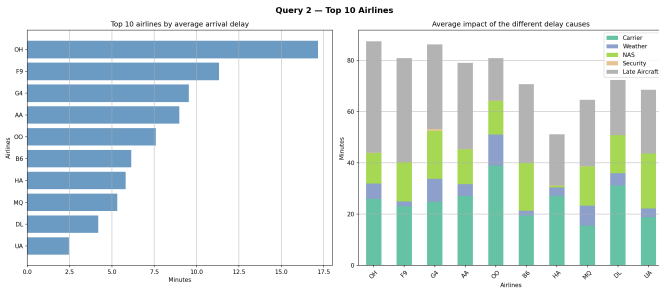


Fig. 2. Top 10 airlines by average arrival delay and the breakdown of their delay root causes.

When evaluating the top 10 airlines with the highest average arrival delays, the data identifies OH as the most delayed carrier, averaging over 17 minutes of arrival delay per flight. The stacked bar chart provides crucial insights into the root causes of these delays. Across almost all carriers in this top 10, the most significant contributing factors are "Late Aircraft" and "Carrier" delays. This suggests that cascading delays from previous flights and internal airline operations are far more disruptive on average than external factors like weather or security. We can also notice that F9 have absolutely no delay caused by security.

C. Query 3

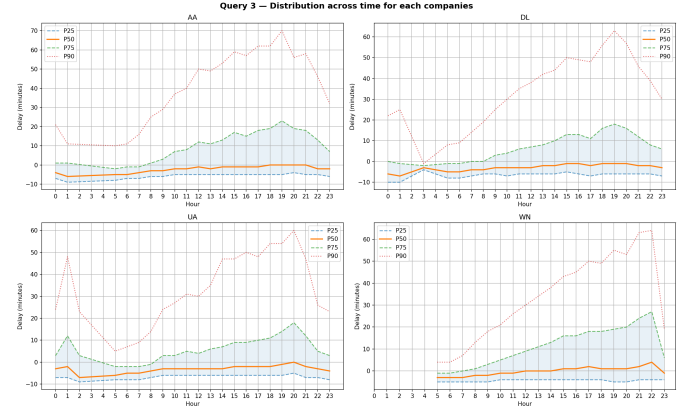


Fig. 3. Hourly distribution of departure delays across the day for AA, DL, UA, and WN, highlighting the 25th, 50th, 75th, and 90th percentiles.

The percentile-based hourly analysis reveals a strong temporal pattern in departure delays for AA, DL, UA, and WN. For all four airlines, the median delay (P50) remains relatively flat and close to zero throughout the entire day, indicating that the majority of flights depart on time. However, the 90th percentile (P90) line demonstrates a sharp, exponential increase starting from the late afternoon and peaking between 18:00 and 21:00. This visualizes the compounding "snowball effect" of delays: while most flights are unaffected, the minority of delayed flights suffer increasingly severe delays as the day progresses.

D. Query 4

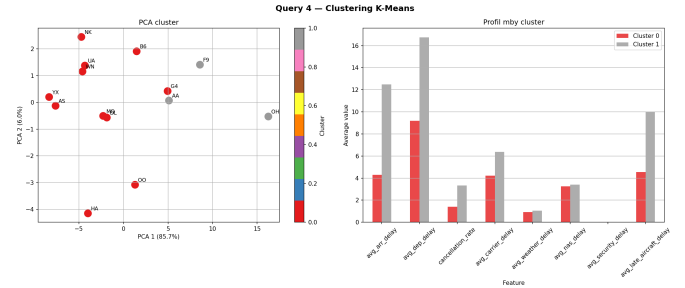


Fig. 4. K-Means clustering ($k = 2$) of airlines based on operational metrics. Left: 2D PCA projection. Right: Average feature profile per cluster.

The K-Means clustering model ($k = 2$) successfully segmented the airlines into two distinct performance profiles. The PCA scatter plot illustrates a clear boundary between the clusters, with the first principal component capturing 85.7% of the total variance. The accompanying bar chart profiling the cluster centroids provides the interpretation: Cluster 1 (gray) groups underperforming airlines characterized by significantly higher metrics across the board, notably in average arrival delays, departure delays, and late aircraft delays. Conversely, Cluster 0 (red) represents the more reliable airlines with lower overall delays and better operational stability.

V. PERFORMANCE BENCHMARKS

To ensure the reliability of our performance metrics and mitigate JVM warm-up overhead, each query was executed over multiple iterations. Our application architecture explicitly separates the transformation phase (Computation) from the action phase (Output to HDFS). The performance results, executed on our local Dockerized cluster, are summarized in Table I.

TABLE I
AVERAGE EXECUTION TIMES PER QUERY (IN MILLISECONDS)

Query	Iteration 1 (ms)	Iteration 2 (ms)	Iteration 3 (ms)
Query 1	9978	2325	2282
Query 2	7663	1896	1589
Query 3	15557	6275	6616
Query 4	17575	7762	6255
Average	12693	4565	4186

The performance evaluation highlights the direct impact of algorithmic complexity on execution times. Queries 1 and 2, which primarily rely on standard filtering and basic mathematical aggregations (e.g., averages, minimums, maximums), demonstrate the fastest execution times.

In contrast, Query 3 exhibits a significant increase in both computation and output duration. This overhead is directly attributed to the calculation of approximate percentiles, an operation that requires extensive data shuffling and sorting across the Spark cluster nodes.

Finally, Query 4 proves to be the most computationally intensive workload. The transition from simple aggregations to an iterative machine learning model requires multiple passes over the data. The feature engineering process (vector assembly and standardization) combined with the K-Means distance calculations drastically increases the CPU load compared to standard DataFrame operations.

Furthermore, it is important to note the significant decrease in execution time observed across successive iterations during our testing phase. The initial run of each query invariably recorded the highest latency. This behavior is inherent to the Java Virtual Machine (JVM) and the Apache Spark architecture. During the first iteration, the system undergoes a "warm-up" phase: the JVM performs ClassLoading and Just-In-Time (JIT) compilation, while Spark initializes the tasks, builds the execution plans (DAGs), and caches the necessary data partitions in RAM. In subsequent iterations, the engine bypasses these initialization steps and leverages the pre-cached data, resulting in drastically reduced and stabilized execution times. Consequently, the average times reported in this benchmark accurately reflect the optimal, steady-state performance of our cluster.

VI. CONCLUSION

This project demonstrated the design and implementation of a distributed Big Data architecture for large-scale flight data analysis. By leveraging Docker Compose, we simulated a complete cluster environment comprising HDFS for resilient storage and Apache Spark for distributed computation.

Through the development of a custom data pipeline in Java, we executed four complex analytical workloads using the typed Dataset API, encompassing temporal aggregations, root cause analysis of delays, and unsupervised machine learning (K-Means clustering). The performance evaluation highlighted the capability of our system to process millions of records efficiently on a standalone node.

For future improvements, several avenues could be explored to extend this work. First, implementing the analytical workloads using Spark SQL would allow for a direct performance comparison against our current Dataset API approach. Second, migrating from a single-node Dockerized simulation to a true physical multi-node cluster would provide valuable insights into the real-world network overhead and distributed scaling capabilities of both HDFS and Spark. Finally, integrating a robust workflow orchestration tool, such as Apache Airflow, would streamline and fully automate the end-to-end data pipeline, from raw data ingestion to the final export of query results instead of a basic setup.sh.